# Other logic gates

Abhilash Jindal

# Other logic operations

- For n variables, $2^{2^n}$ boolean functions are possible

**Table 2.7**
*Truth Tables for the 16 Functions of Two Binary Variables*

| $x$ | $y$ | $F_0$ | $F_1$ | $F_2$ | $F_3$ | $F_4$ | $F_5$ | $F_6$ | $F_7$ | $F_8$ | $F_9$ | $F_{10}$ | $F_{11}$ | $F_{12}$ | $F_{13}$ | $F_{14}$ | $F_{15}$ |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 1 | 1 | 1 | 1 | 1 | 1 | 1 | 1 |
| 0 | 1 | 0 | 0 | 0 | 0 | 1 | 1 | 1 | 1 | 0 | 0 | 0 | 0 | 1 | 1 | 1 | 1 |
| 1 | 0 | 0 | 0 | 1 | 1 | 0 | 0 | 1 | 1 | 0 | 0 | 1 | 1 | 0 | 0 | 1 | 1 |
| 1 | 1 | 0 | 1 | 0 | 1 | 0 | 1 | 0 | 1 | 0 | 1 | 0 | 1 | 0 | 1 | 0 | 1 |

# Boolean Functions

- Two constant functions: $F_0 = 0; F_{15} = 1$
- AND/OR: $F_1 = xy; F_7 = x + y$
- Transfer: $F_3 = x; F_5 = y$
- Complement: $F_{10} = y'; F_{12} = x'$

# Other logic operations

- Inhibition:
  - $F_2 = xy'$ . x is true and y is not true
  - $F_4 = x'y$ . y is true and x is not true
- Implication:
  - $F_{11} : x + y' = x \subset y$. When y is true, x must be true.
  - $F_{13} : x' + y = x \supset y$. When x is true, y must be true.
- Used by logicians. $rain \subset wet$. Not much in digital logic as they are **not commutative**

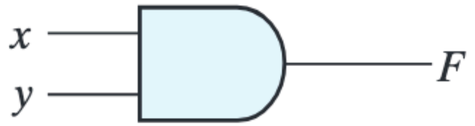
# Other logic operations

- Can write each one using ANDs, ORs, and NOTs
- Four remaining functions:
  - NAND: Not of AND
  - NOR: Not of OR
  - XOR: Are x and y not equal?
  - Equivalence: Are x and y equal? XNOR.

| Boolean Functions | Operator Symbol | Name | Comments |
|---|---|---|---|
| $F_0 = 0$ | | Null | Binary constant 0 |
| $F_1 = xy$ | $x \cdot y$ | AND | $x$ and $y$ |
| $F_2 = xy'$ | $x/y$ | Inhibition | $x$, but not $y$ |
| $F_3 = x$ | | Transfer | $x$ |
| $F_4 = x'y$ | $y/x$ | Inhibition | $y$, but not $x$ |
| $F_5 = y$ | | Transfer | $y$ |
| $F_6 = xy' + x'y$ | $x \oplus y$ | Exclusive-OR | $x$ or $y$, but not both |
| $F_7 = x + y$ | $x + y$ | OR | $x$ or $y$ |
| $F_8 = (x + y)'$ | $x \downarrow y$ | NOR | Not-OR |
| $F_9 = xy + x'y'$ | $(x \oplus y)'$ | Equivalence | x equals $y$ |
| $F_{10} = y'$ | $y'$ | Complement | Not $y$ |
| $F_{11} = x + y'$ | $x \subset y$ | Implication | If $y$, then $x$ |
| $F_{12} = x'$ | $x'$ | Complement | Not $x$ |
| $F_{13} = x' + y$ | $x \supset y$ | Implication | If $x$, then $y$ |
| $F_{14} = (xy)'$ | $x \uparrow y$ | NAND | Not-AND |
| $F_{15} = 1$ | | Identity | Binary constant 1 |

# Logic gates

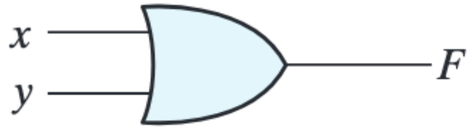
AND




$F = x \cdot y$

| $x$ | $y$ | $F$ |
|---|---|---|
| 0 | 0 | 0 |
| 0 | 1 | 0 |
| 1 | 0 | 0 |
| 1 | 1 | 1 |

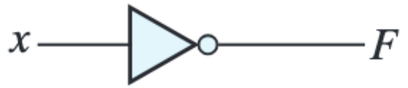
OR




$F = x + y$

| $x$ | $y$ | $F$ |
|---|---|---|
| 0 | 0 | 0 |
| 0 | 1 | 1 |
| 1 | 0 | 1 |
| 1 | 1 | 1 |

Inverter




$F = x'$

| $x$ | $F$ |
|---|---|
| 0 | 1 |
| 1 | 0 |

Buffer

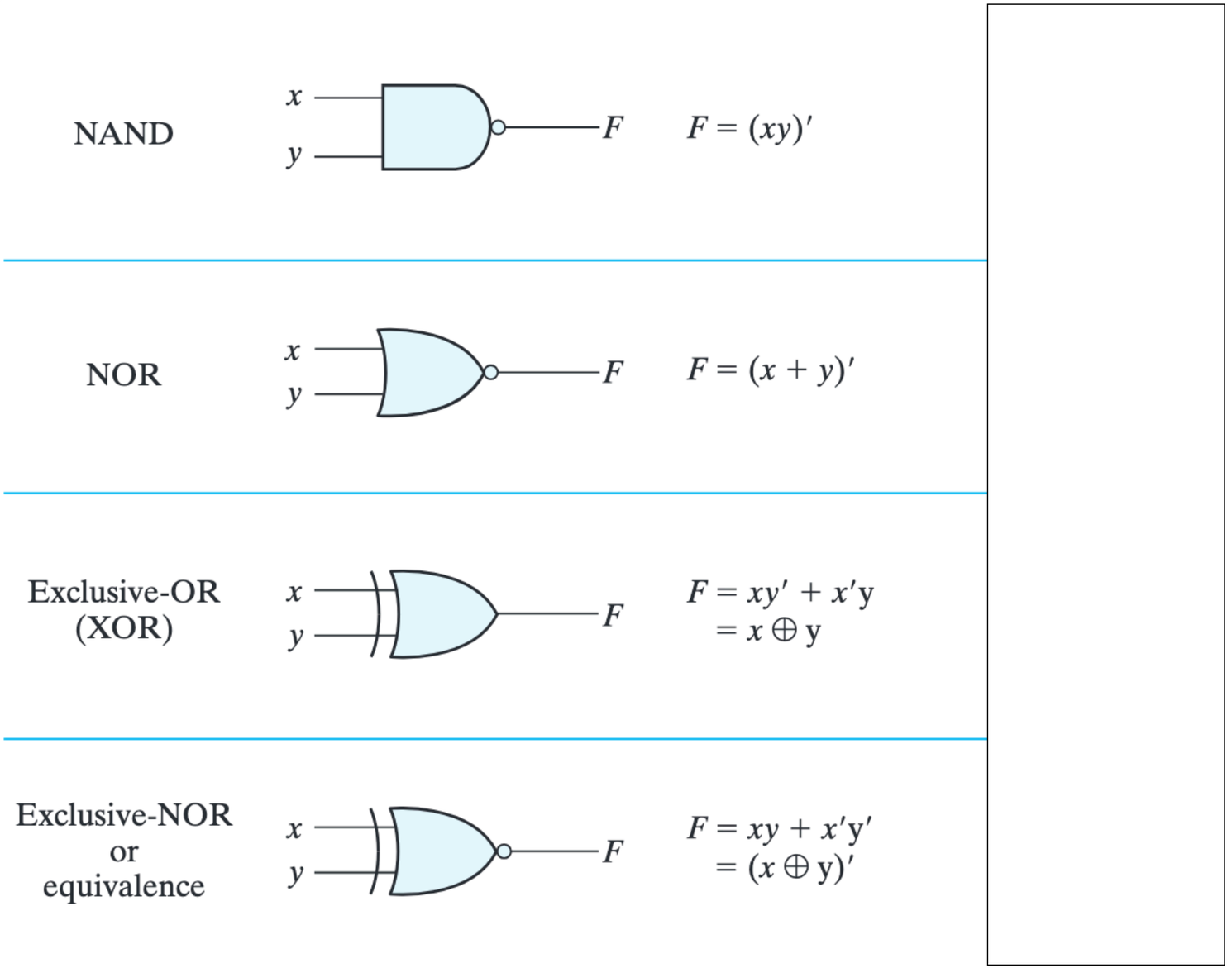



$F = x$

| $x$ | $F$ |
|---|---|
| 0 | 0 |
| 1 | 1 |

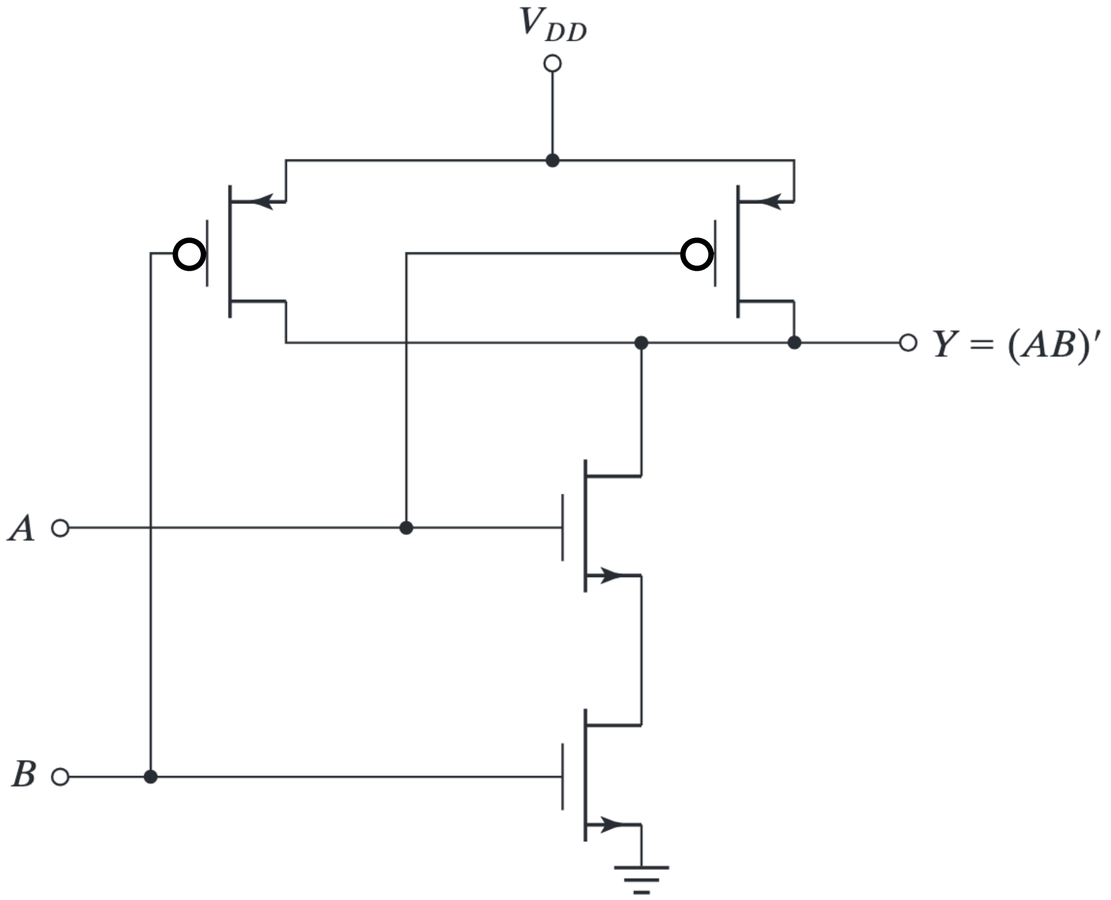
# Other logic gates

- XOR adds another line to OR
- NAND, NOR, XNOR add a bubble
- All of them are commutative



# NAND gate is easy to construct from transistors

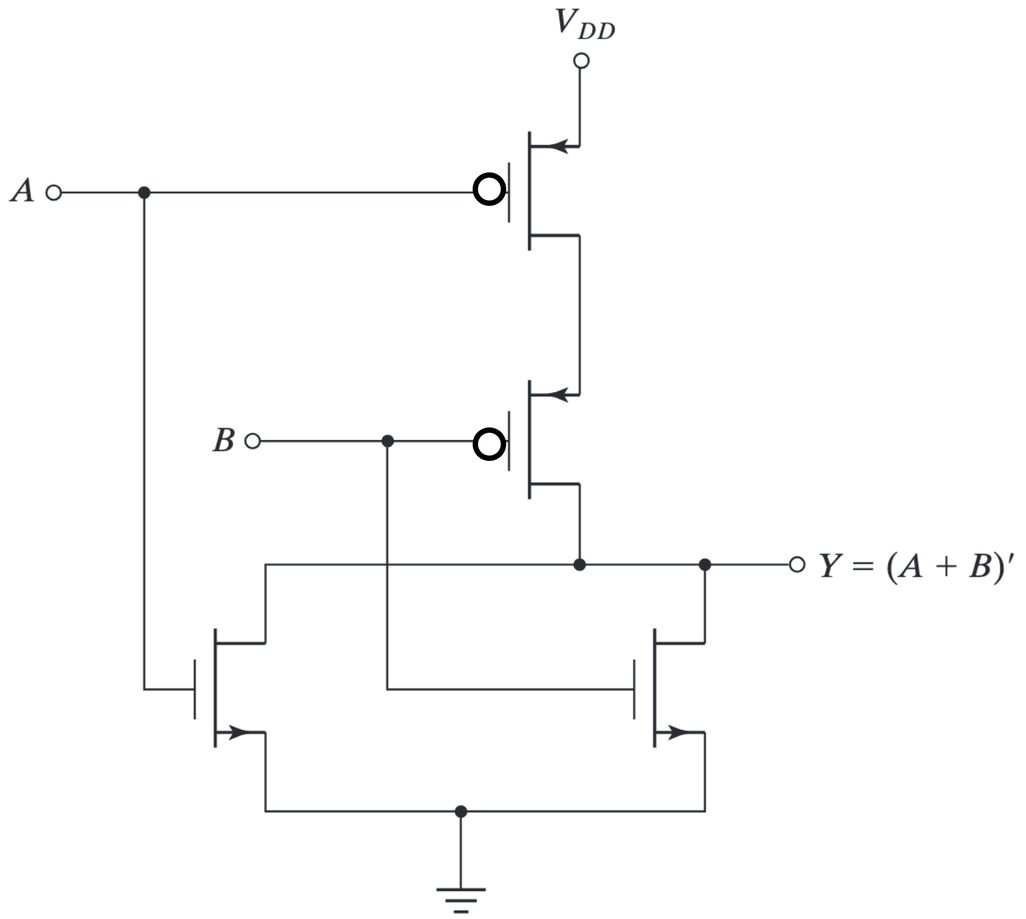
- When $A = GND$ and $B = GND$, $Y = V_{DD}$
- When $A = GND, B = V_{DD}$. $Y = V_{DD}$




(b) NAND gate

# NOR gate is easy to construct from transistors

- When $A = GND$ and $B = GND$, $Y = V_{DD}$
- When $A = GND, B = V_{DD}$. $Y = GND$
- AND, OR, and XOR each require 6 transistors, instead of 4 for NOR and NAND




(c) NOR gate

# Extension to multiple inputs

## AND and OR

- AND and OR could be easily extended as they are both associative and commutative

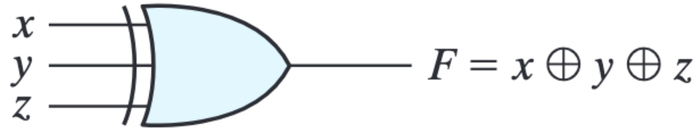
- $x + y = y + x;$
  $x + (y + z) = (x + y) + z$

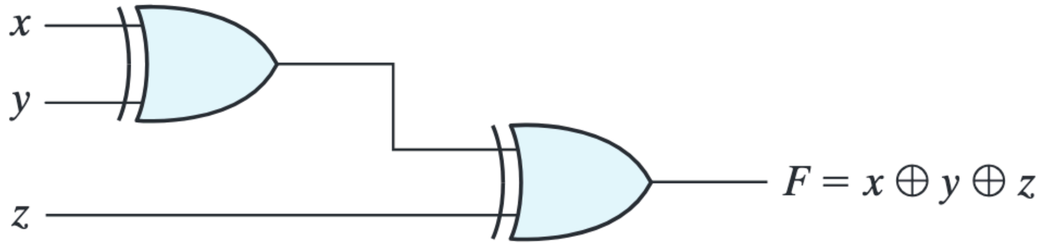
- $x \,.\, y = y \,.\, x;$
  $x \,.\, (y \,.\, z) = (x \,.\, y) \,.\, z$

# Extension to multiple inputs

## XOR and XNOR

- XOR and XNOR are associative and commutative
- $(xy' + x'y)'z + (xy' + x'y)z'$
- $(x'y' + xy)z + (xy' + x'y)z'$
- $x'y'z + xyz + xy'z' + x'yz'$
- $x'(y'z + yz') + x(yz + y'z')$
- $x'(y'z + yz') + x(y'z + yz')'$



# Extension to multiple inputs

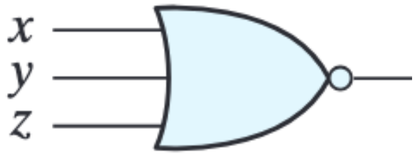
## Difficulty with NOR and NAND

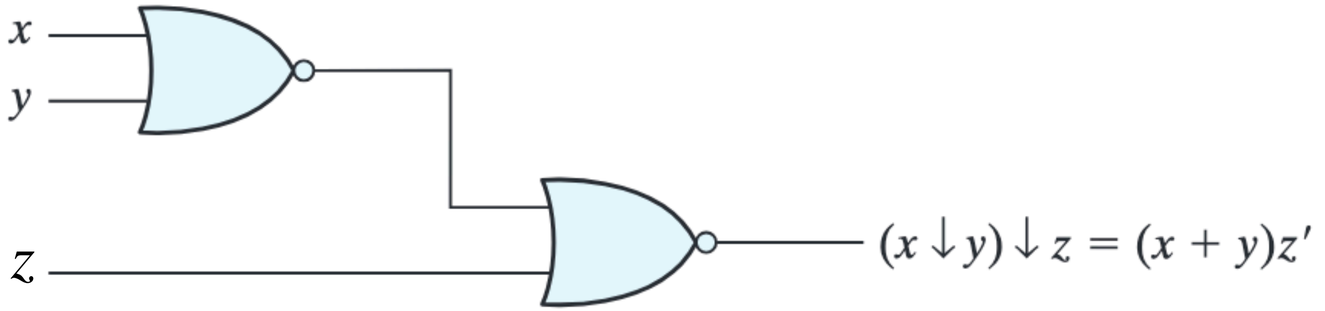
- NOR
  - $[(x + y)' + z]' = (x + y)z' = xz' + yz'$
  - $[x + (y + z)']' = x'(y + z)' = x'y + x'z$
- NAND
  - $[(xy)'z]' = xy + z'$
  - $[x(yz)']' = x' + yz$

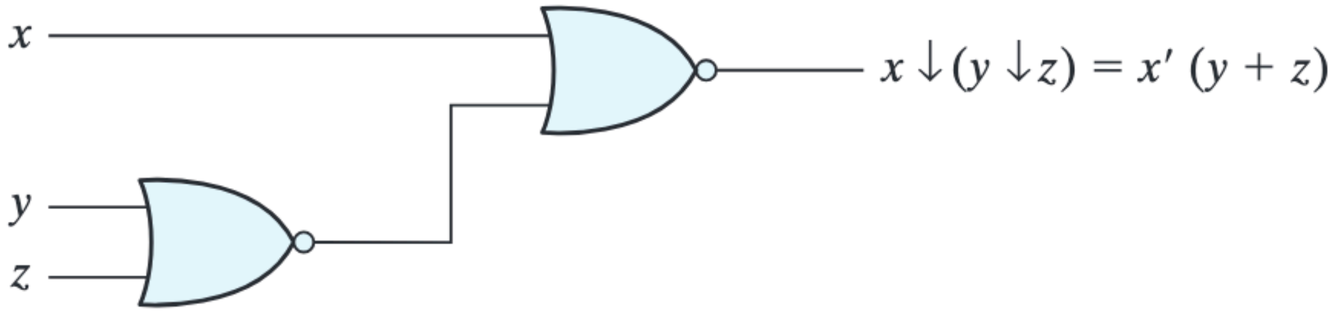
# Extension to multiple inputs

## Difficulty with NOR (2)

- Which circuit is this equivalent to?



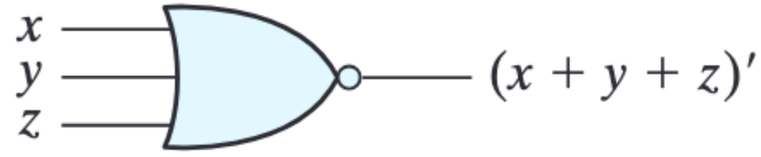



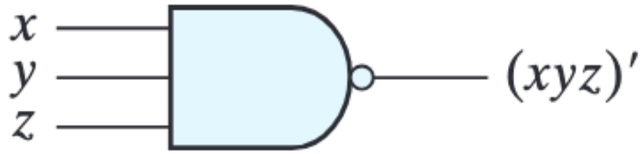
# Extension to multiple inputs

## NOR and NAND

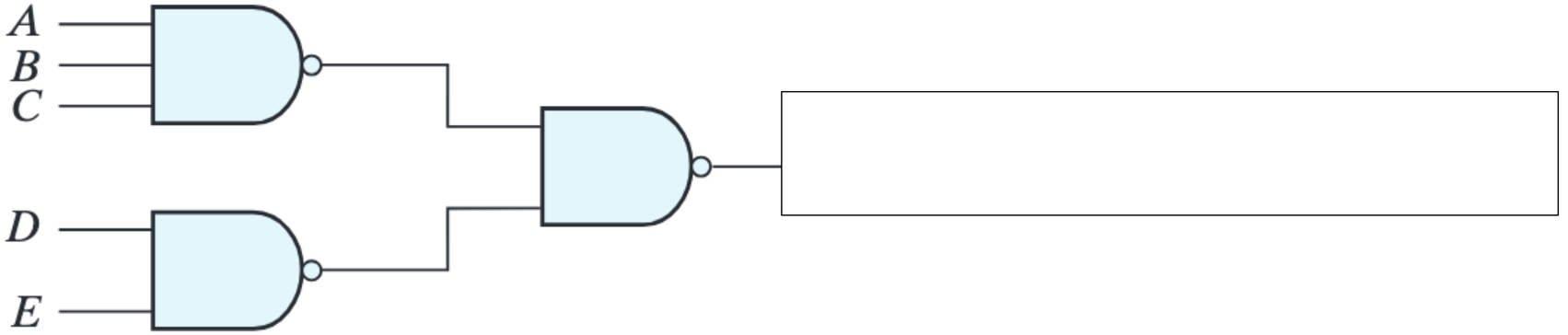
- Treat NOR as “complemented OR” since OR is associative




(a) 3-input NOR gate

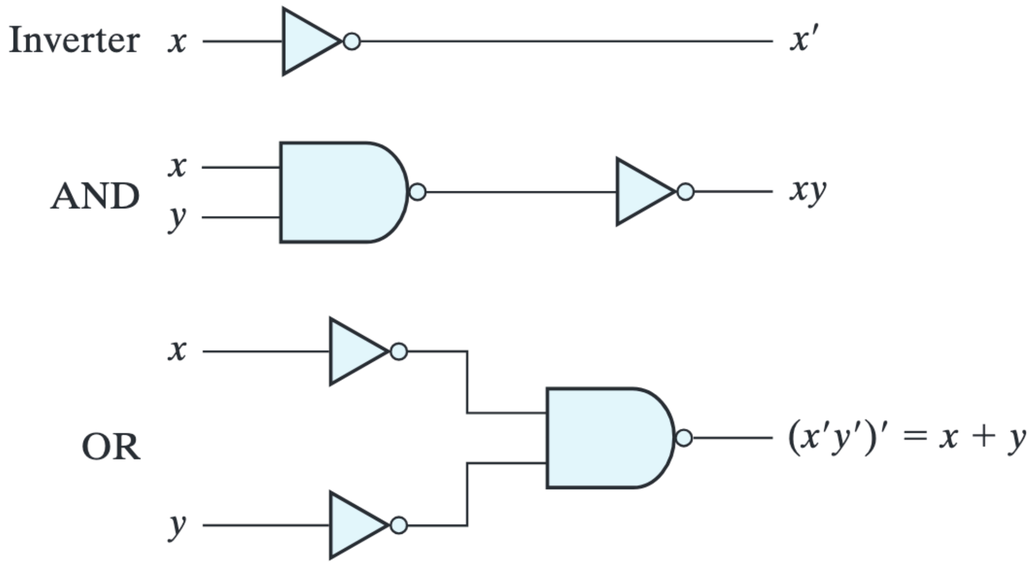



(b) 3-input NAND gate



# NAND gate is *universal gate*

- Any function can be implemented with AND, OR, NOT gates (sum-of-products/product-of-sums)



# Convert functions to NAND circuit

- Convert AND-OR diagrams (sum-of-products) to NAND diagrams

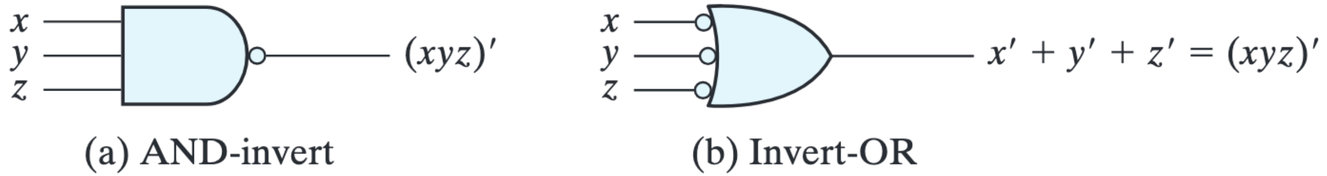



**FIGURE 3.17**
**Two graphic symbols for a three-input NAND gate**

# Convert sum-of-product functions to NAND circuit

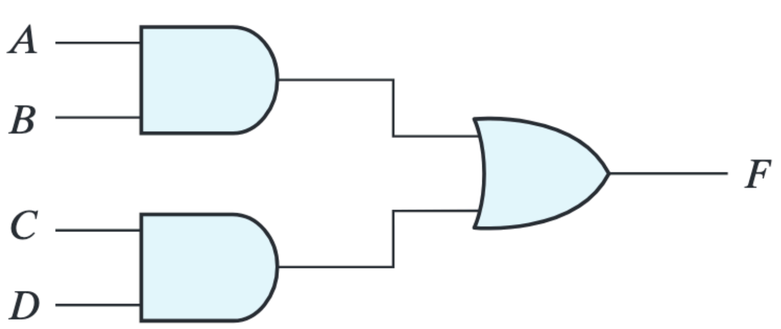
## F = AB + CD

1. Replace AND gates with AND-invert gate. OR with Invert-OR gate

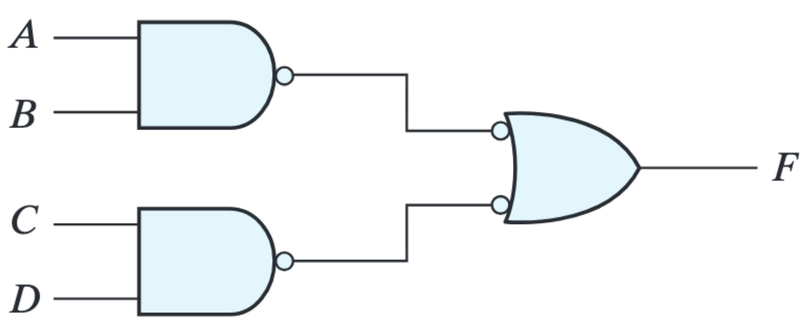
2. Two bubbles represent double-complement: equal to no-complement

3. Convert Invert-OR to AND-invert

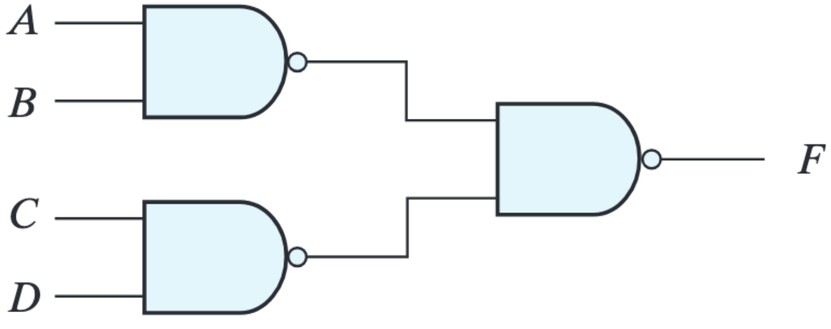
F = ((AB)'(CD)')' = AB + CD




(a)




(b)

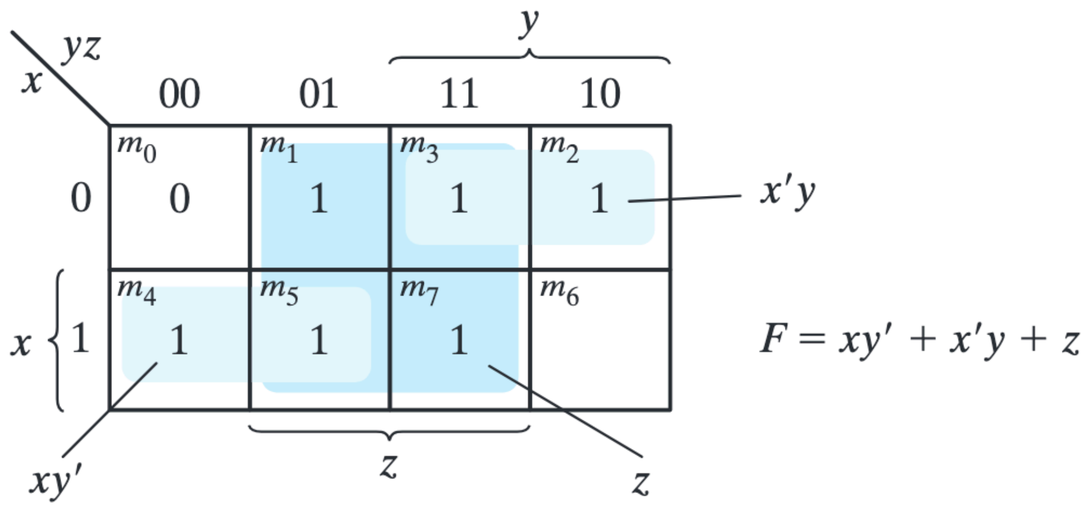


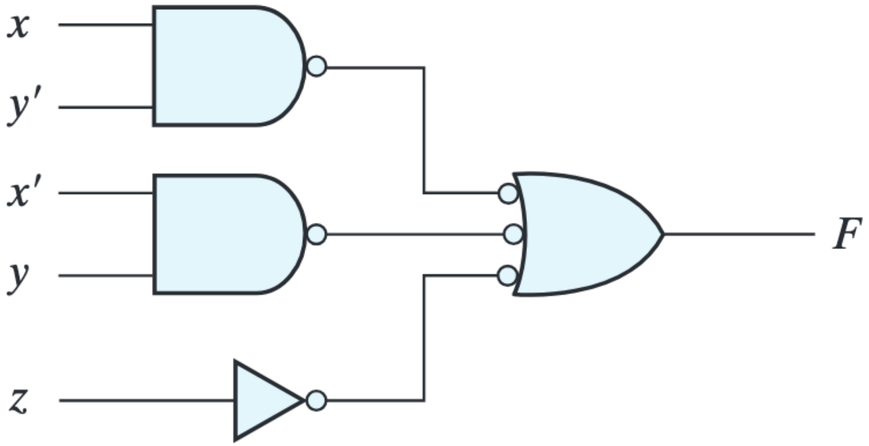

(c)

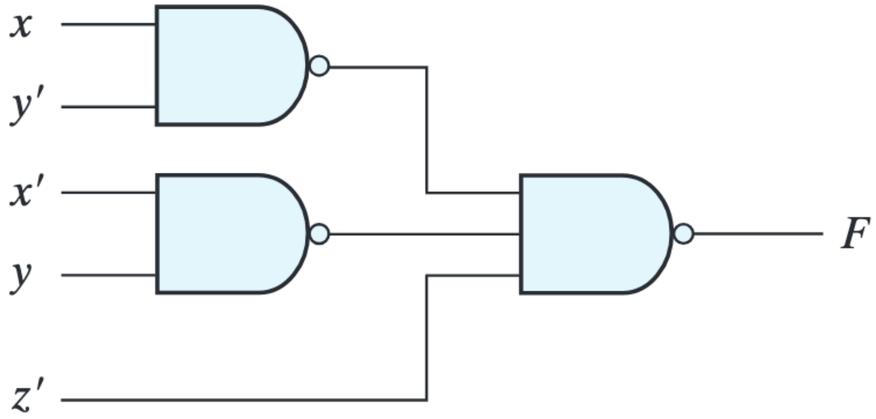
# Convert canonical functions to NAND circuit

$F(x, y, z) = \Sigma(1,2,3,4,5,7)$

- Notice that we need to invert z



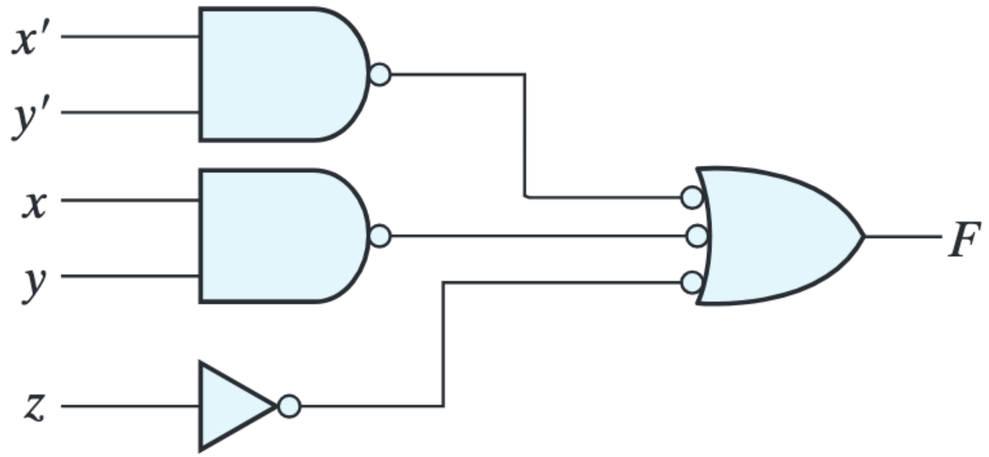



# Convert canonical functions to NAND circuit

$F(x, y, z) = \Sigma(0,1,3,5,6,7)$

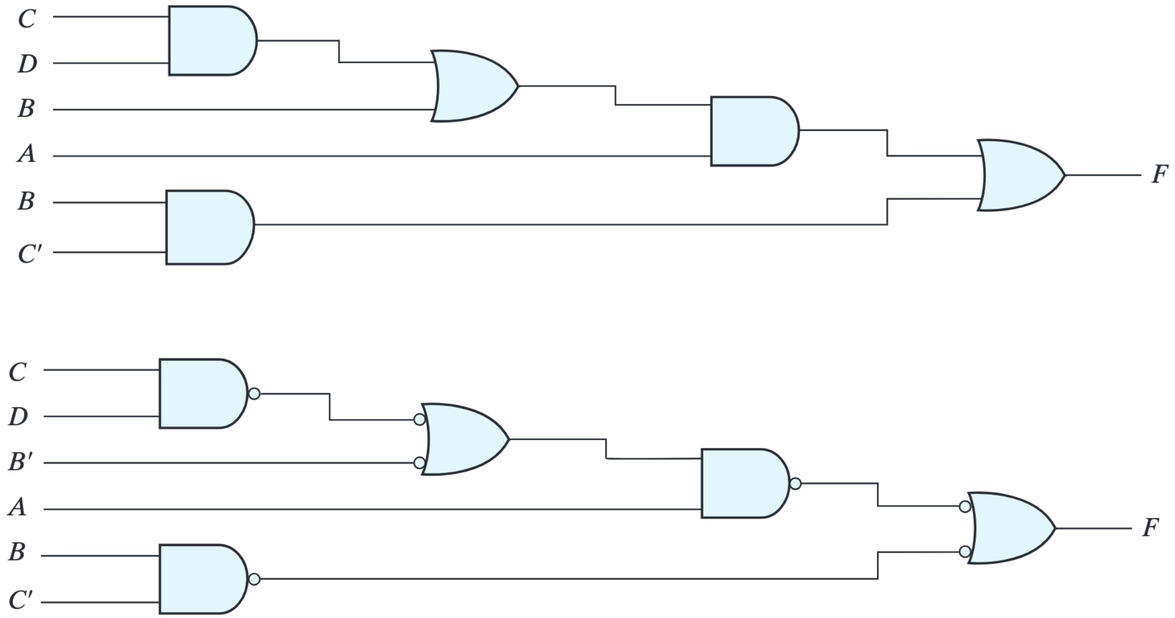
- $F(x, y, z) = x'y' + xy + z$



# Multilevel NAND circuits

## F = A(CD+B) + BC’

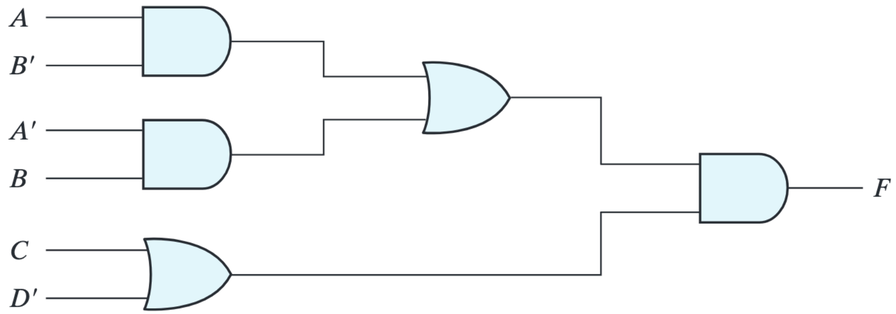
- Change AND to AND-invert, OR to invert-OR
- For every bubble that is not compensated, introduce inverter (e.g., B -> B’)

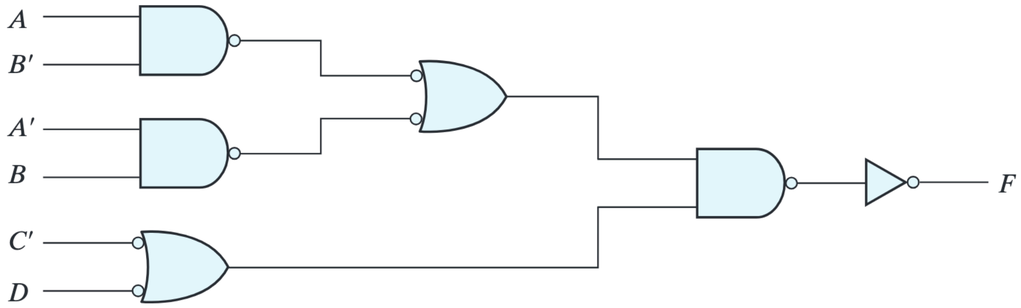


# Multilevel NAND circuits

## F = (AB’+A’B)(C+D’)

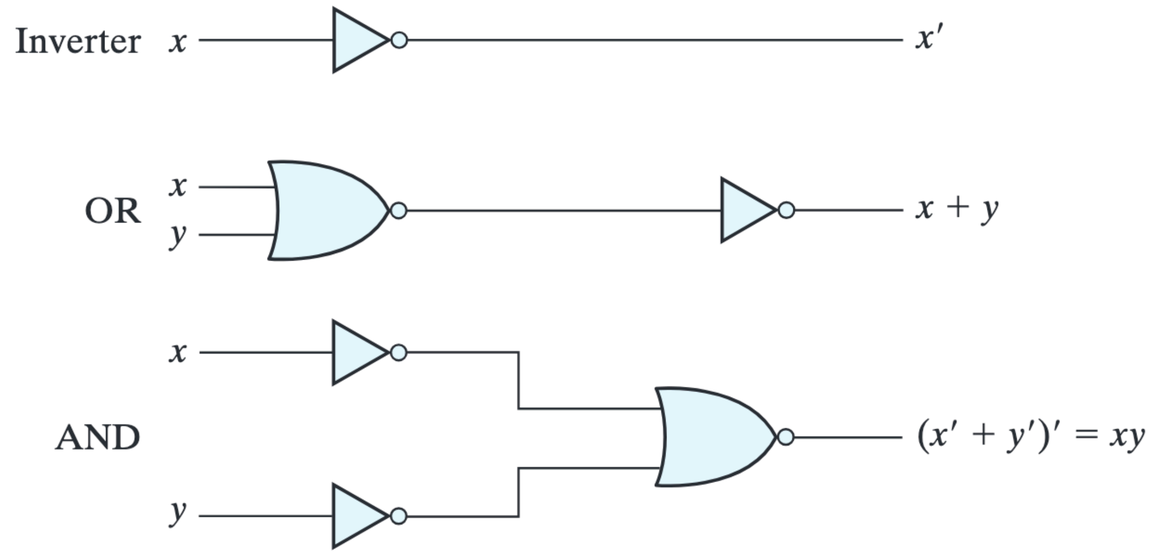
- Change AND to AND-invert, OR to invert-OR
- For every bubble that is not compensated, introduce inverter (e.g., C -> C’, D’ -> D, before F)



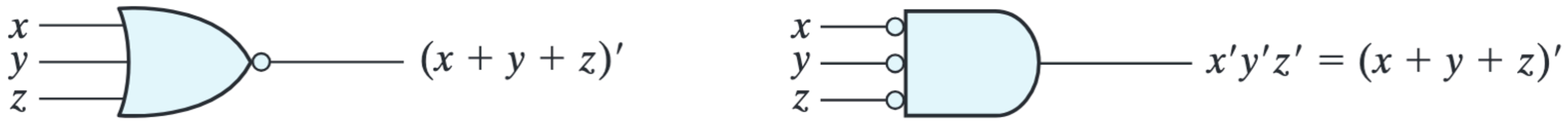
# NOR gate is a *universal gate*

- Any function can be implemented with AND, OR, NOT gates (sum-of-products/product-of-sums)



# Convert functions to NOR circuit

- Convert OR-AND diagrams (product-of-sums) to NOR diagrams




(a) OR-invert (b) Invert-AND

**FIGURE 3.23**
**Two graphic symbols for the NOR gate**

# Convert product-of-sums functions to NOR circuit

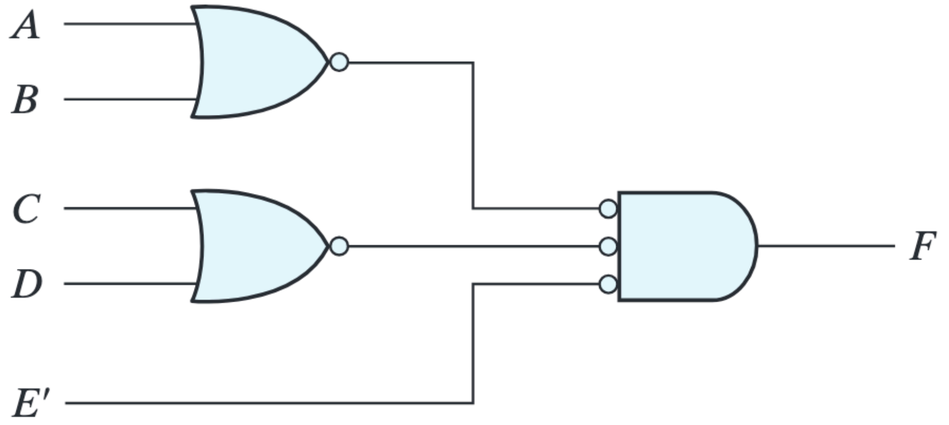
$F = (A + B)(C + D)E$

1. Replace OR gates with OR-invert gate. AND with Invert-AND gate

2. Two bubbles represent double-complement: equal to no-complement

3. Convert Invert-AND to OR-invert.

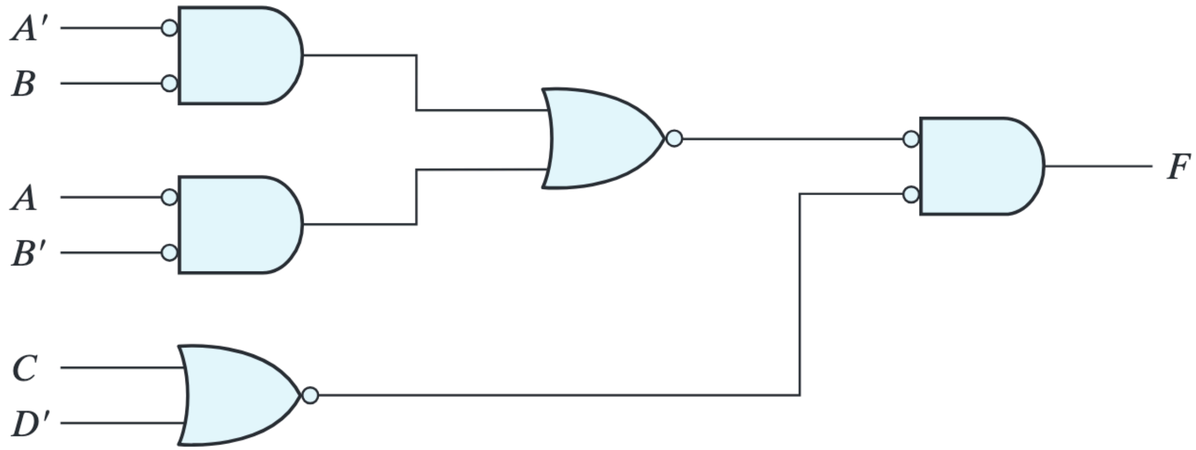
Notice that we need to invert E



# Convert product-of-sums functions to NOR circuit

$F = (AB' + A'B)(C + D')$

- Need to invert A, B', A', B



# Other two-level implementations

## AND, OR, NAND, NOR

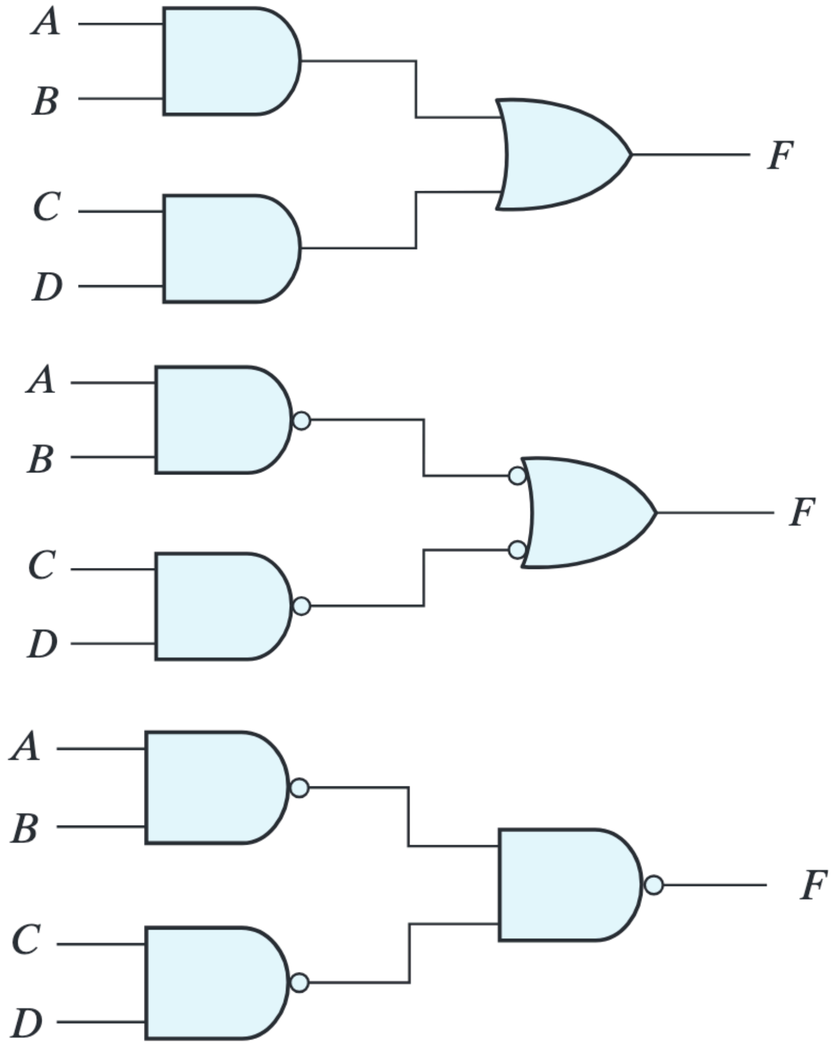
Degenerate to a single gate

- AND-AND: (AB)(CD)
- OR-OR: (A+B)+(C+D)
- OR-NOR/NOR-OR: ((A+B)+(C+D))’
- AND-NAND/NAND-AND: ((AB)(CD))’

# Other two-level implementations

## AND, OR, NAND, NOR

- AND-OR <> NAND-NAND
  - Sum of products
- OR-AND <> NOR-NOR
  - Product of sums
- NAND-AND <> AND-NOR:
  - AND-INVERT-AND <> AND-OR-INVERT
- NOR-OR <> OR-NAND
  - OR-INVERT-OR <> OR-AND-INVERT

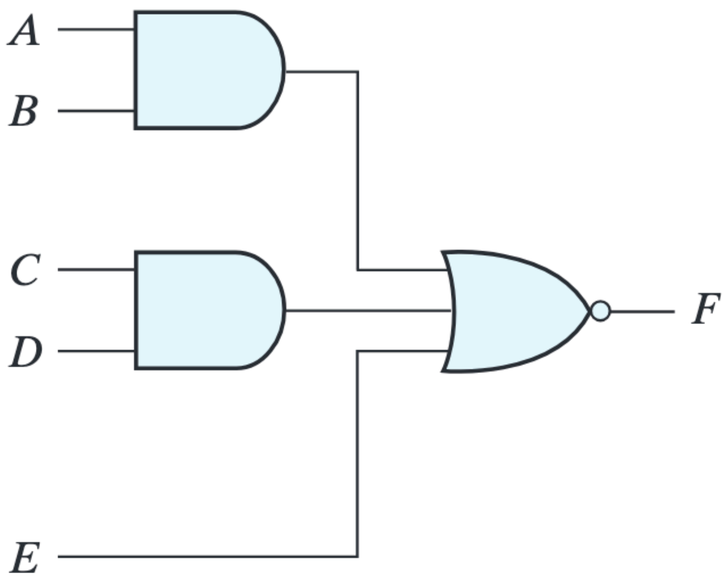


# AND-OR-INVERT

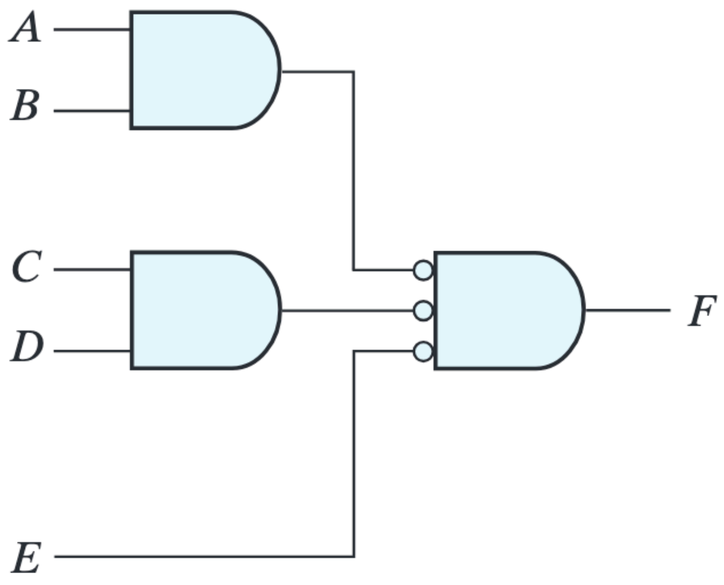
## AND-NOR <> NAND-AND

- Push the bubble back

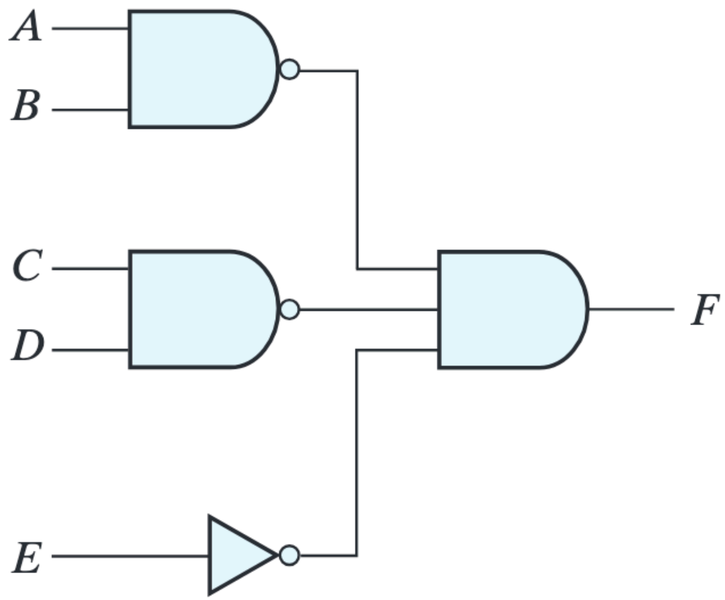
$$F = (AB + CD + E)'$$




(a) AND–NOR




(b) AND–NOR



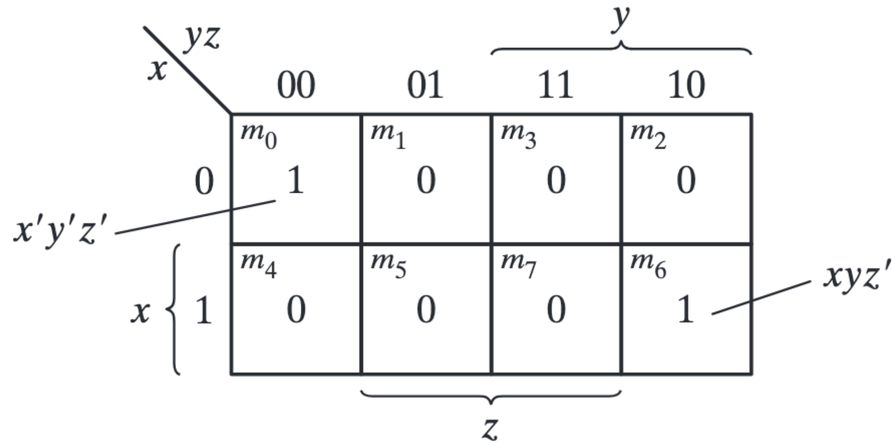

(c) NAND–AND

# Convert function to AND-OR-INVERT

## AND-NOR <> NAND-AND

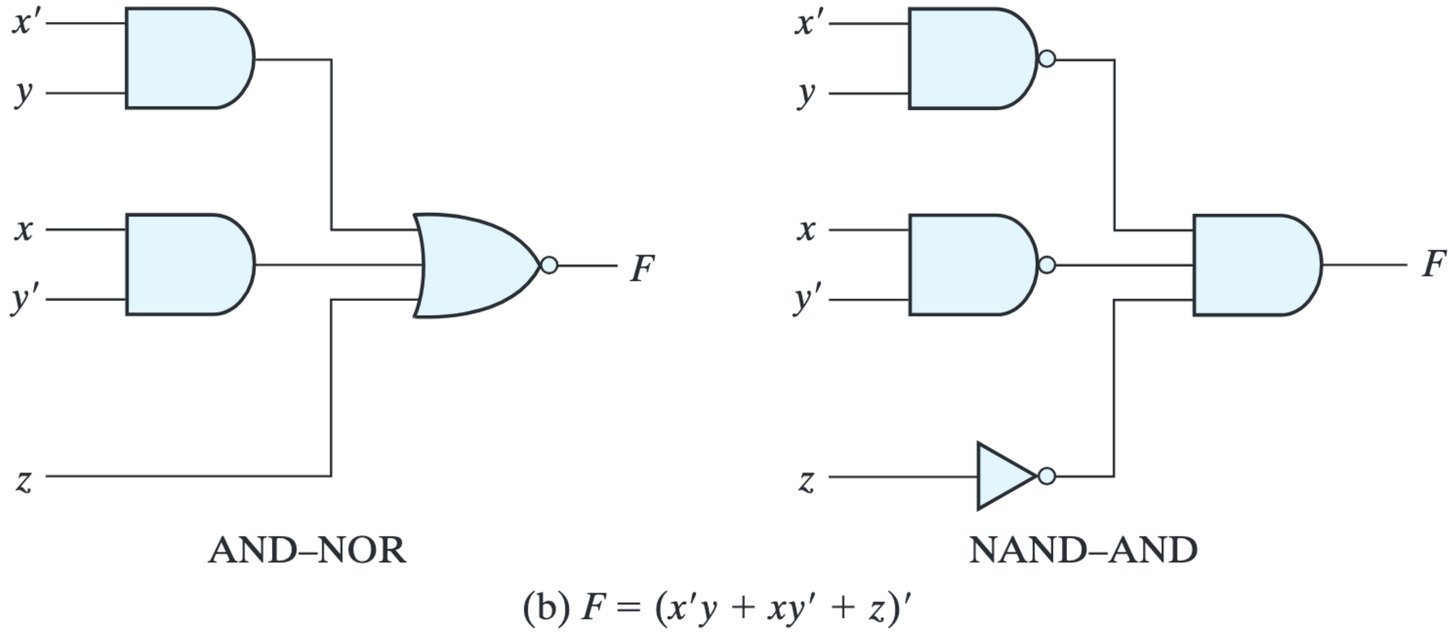
- Merge 0s in K-map to write F’ as sum-of-products form.
- Draw AND-OR diagram for F’. Change OR to NOR to get F

$$F = \Sigma(0,6)$$




$F = x'y'z' + xyz'$

$F' = x'y + xy' + z$



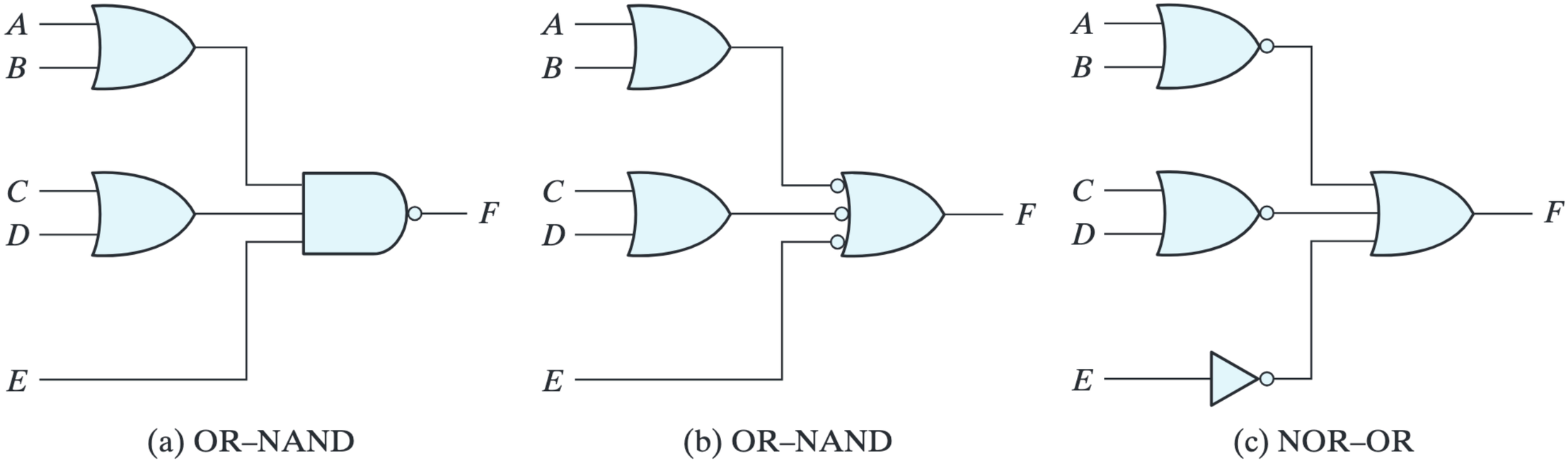

(b) $F = (x'y + xy' + z)'$

# OR-AND-INVERT

## OR-NAND <> NOR-OR

- Push the bubble back

$$F = [(A + B)(C + D)E]'$$

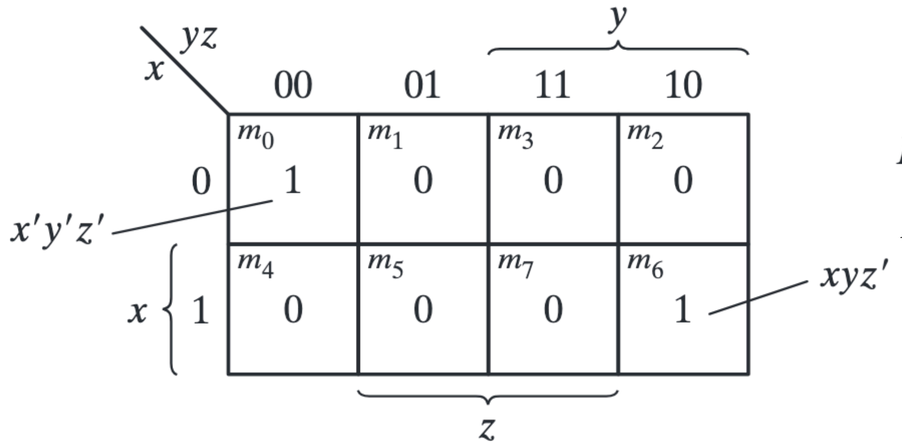



(a) OR–NAND (b) OR–NAND (c) NOR–OR

# Convert function to OR-AND-INVERT

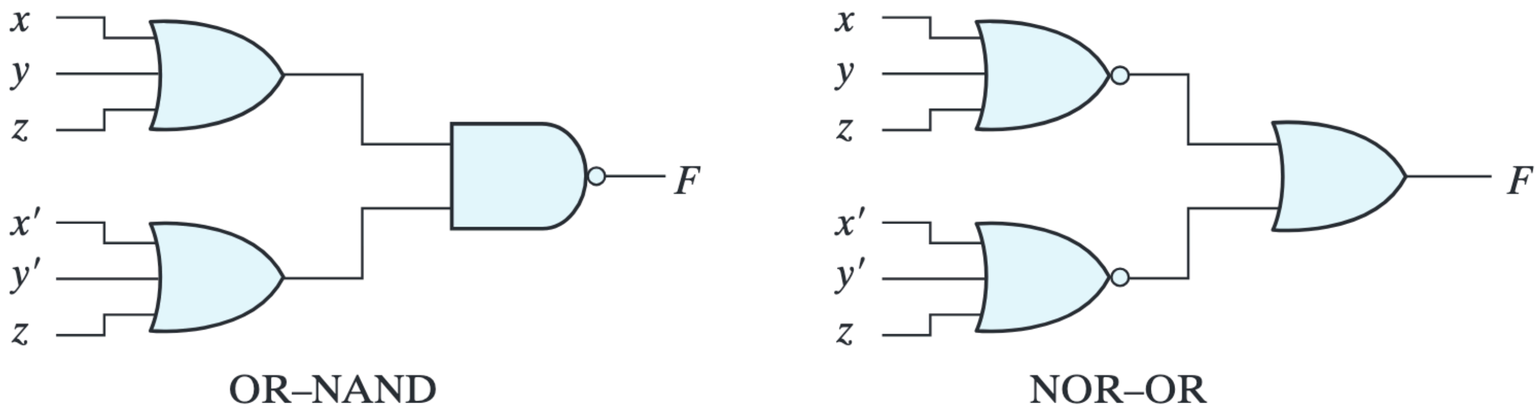
## OR-NAND <> NOR-OR

- Merge 1s in K-map to write F. Write F as sum-of-products form
- Take complement to get F’ in product-of-sums form
- Draw OR-AND diagram for F’. Change AND to NAND to get F




$F = x'y'z' + xyz'$

$F' = [(x + y + z)(x' + y' + z)]$




(c) $F = [(x + y + z)\,(x' + y' + z)]'$

$$F = \Sigma(0,6)$$

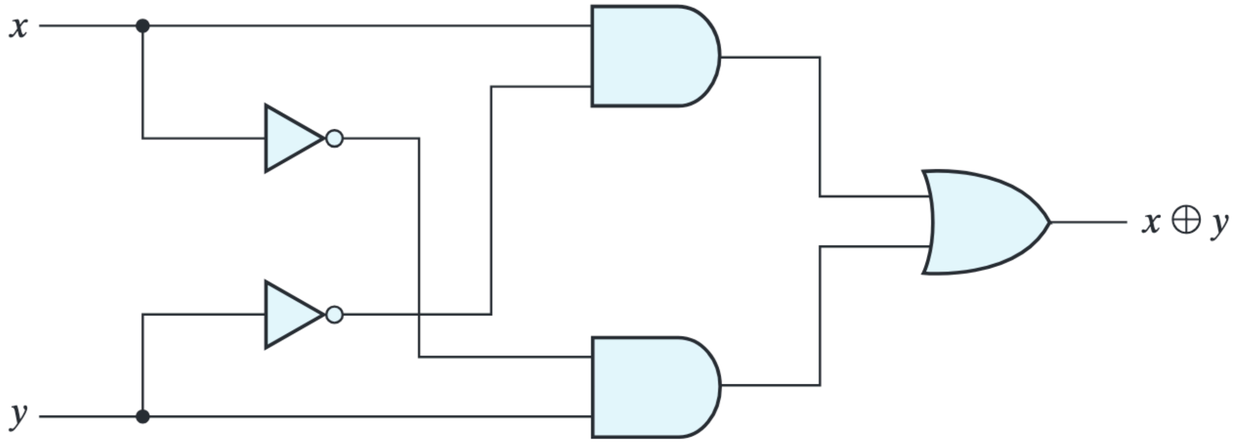
# XOR function

- $x \oplus y = xy' + x'y$. 1 when x and y differ. 0 when equal
- $(x \oplus y)' = (xy' + x'y)' = (x' + y)(x + y') = xy + x'y'$
  - 1 when x and y are equal. 0 when they differ.

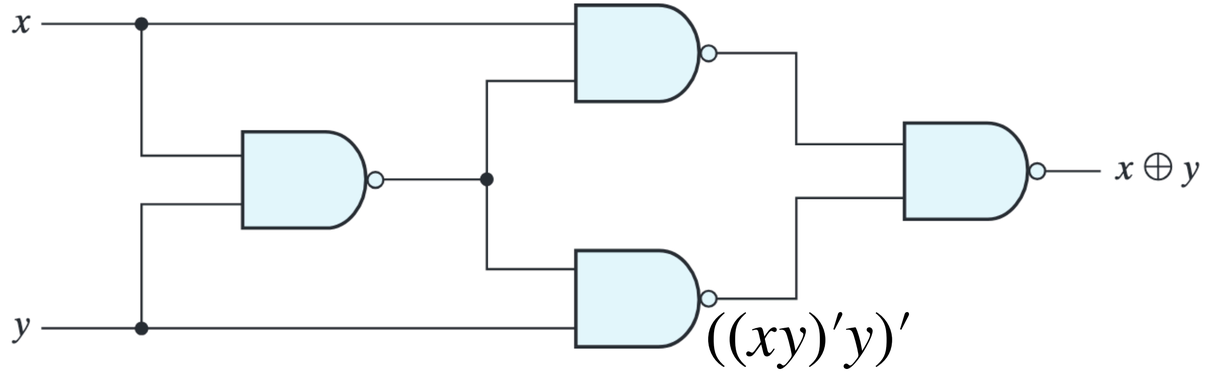
- $x \oplus 0$
  - $x$
- $x \oplus 1$
  - $x'$
- $x \oplus x'$
  - $1$
- $x \oplus x$
  - $0$
- $x \oplus y'$
  - $x' \oplus y = (x \oplus y)'$
- $x \oplus y = y \oplus x$
- $(x \oplus y) \oplus z = x \oplus (y \oplus z)$

# Implementing XOR

- $x \oplus y = xy' + x'y$
- Change AND to AND-INVERT and OR to INVERT-OR. Both are NAND
- Note that $(x'y)'$ is same as $((xy)'y)' = ((x' + y')y')'$. Replace two NOT gates with one NAND gate



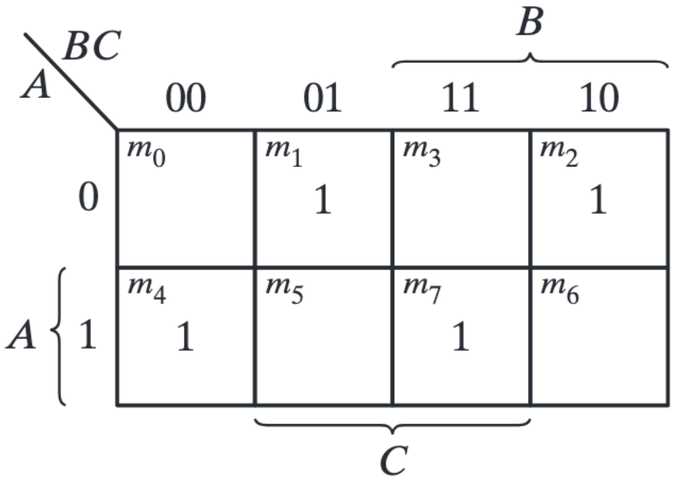

(a) Exclusive-OR with AND–OR–NOT gates



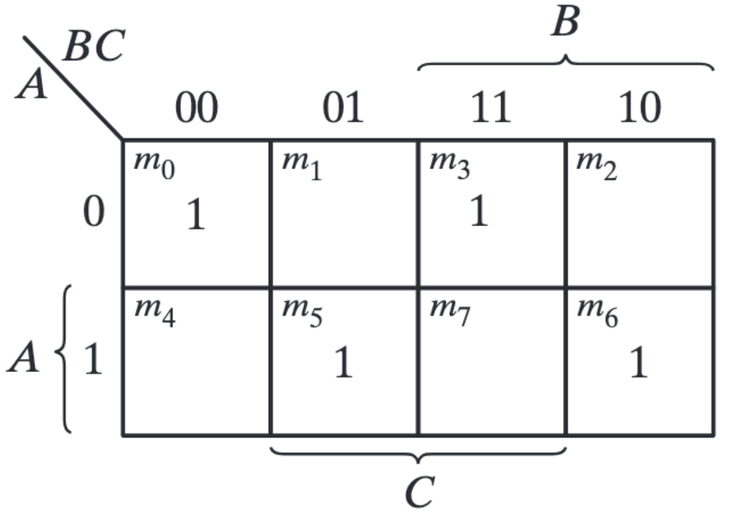

(b) Exclusive-OR with NAND gates

# Error detection with XOR gate

- $A \oplus B \oplus C = (AB' + A'B) \oplus C$
- $= (AB' + A'B)C' + (AB' + A'B)'C$
- $= (AB' + A'B)C' + (AB + A'B')C$
- $= AB'C' + A'BC' + ABC + A'B'C$
- $= \Sigma(1,2,4,7)$
- Odd number of 1s



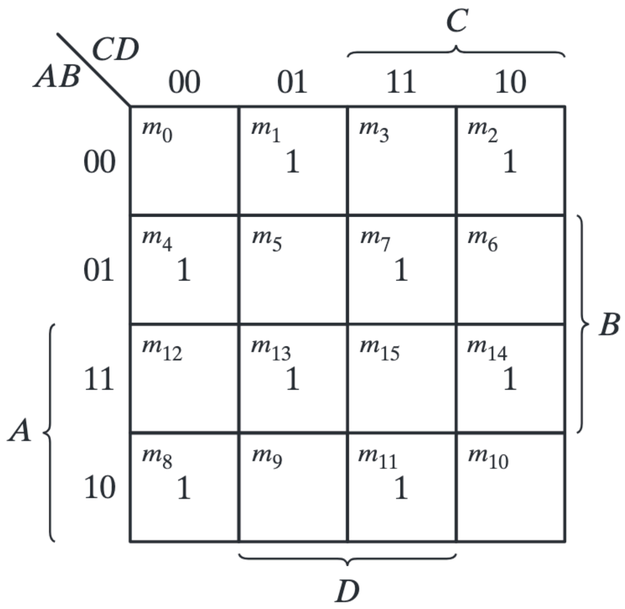

(a) Odd function $F = A \oplus B \oplus C$




(b) Even function $F = (A \oplus B \oplus C)'$

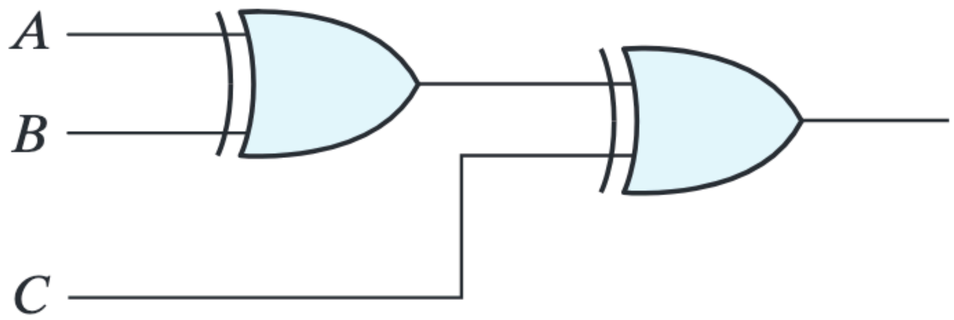
# Four-variable XOR function

- $A \oplus B \oplus C \oplus D = (AB' + A'B) \oplus (CD' + C'D)$
- $= (AB' + A'B)(CD' + C'D)' + (AB' + A'B)'(CD' + C'D)$
- $= (AB' + A'B)(CD + C'D') + (A'B' + AB)(CD' + C'D)$
- $= \Sigma(1,2,4,7,8,11,13,14)$

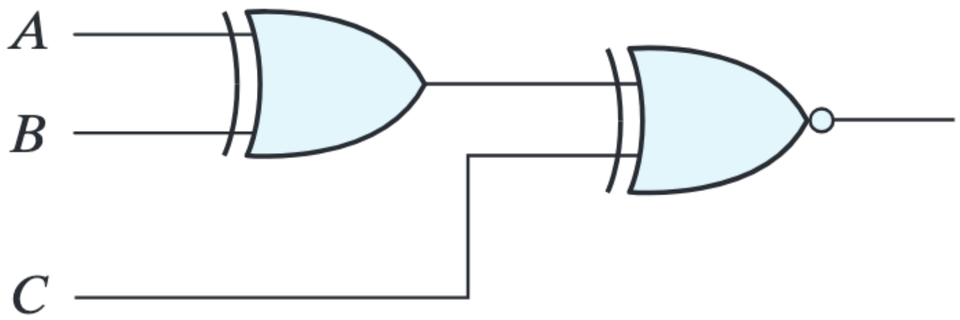



(a) Odd function $F = A \oplus B \oplus C \oplus D$

# Implementing odd and even functions




(a) 3-input odd function




(b) 3-input even function

- $A \oplus B \oplus C \oplus D = (AB' + A'B) \oplus (CD' + C'D)$

# Parity generation

- Parity bit is 1 if odd number of 1s
- $P = x \oplus y \oplus z$
- Transmit parity bit P along with data (x,y,z)

| Three-Bit Message | | | Parity Bit |
|---|---|---|---|
| *x* | *y* | *z* | *P* |
| 0 | 0 | 0 | 0 |
| 0 | 0 | 1 | 1 |
| 0 | 1 | 0 | 1 |
| 0 | 1 | 1 | 0 |
| 1 | 0 | 0 | 1 |
| 1 | 0 | 1 | 0 |
| 1 | 1 | 0 | 0 |
| 1 | 1 | 1 | 1 |

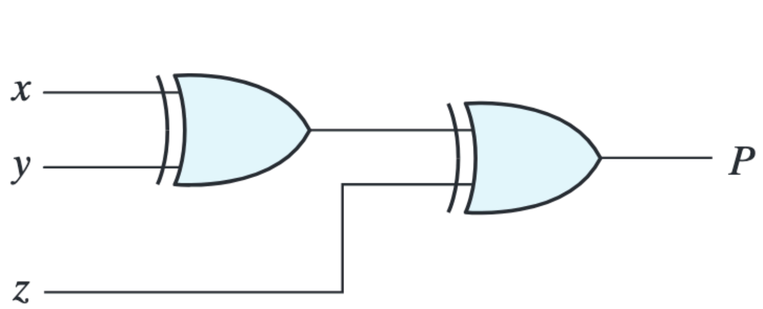
# Parity checking

- Can catch only one-or-three bit flips
- Error if four bits (x,y,z,P) have odd number of 1's
- $C = x \oplus y \oplus z \oplus P$

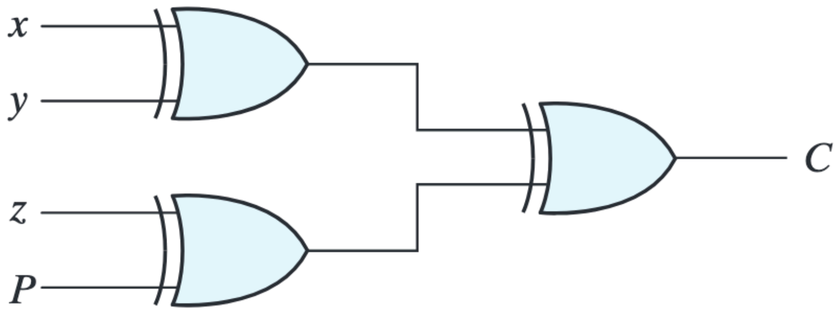
| Four Bits Received | | | | Parity Error Check |
|---|---|---|---|---|
| *x* | *y* | *z* | *P* | *C* |
| 0 | 0 | 0 | 0 | 0 |
| 0 | 0 | 0 | 1 | 1 |
| 0 | 0 | 1 | 0 | 1 |
| 0 | 0 | 1 | 1 | 0 |
| 0 | 1 | 0 | 0 | 1 |
| 0 | 1 | 0 | 1 | 0 |
| 0 | 1 | 1 | 0 | 0 |
| 0 | 1 | 1 | 1 | 1 |
| 1 | 0 | 0 | 0 | 1 |
| 1 | 0 | 0 | 1 | 0 |
| 1 | 0 | 1 | 0 | 0 |
| 1 | 0 | 1 | 1 | 1 |
| 1 | 1 | 0 | 0 | 0 |
| 1 | 1 | 0 | 1 | 1 |
| 1 | 1 | 1 | 0 | 1 |
| 1 | 1 | 1 | 1 | 0 |

# Parity generation and checking

- Sender needs parity generator circuit; receiver needs parity checker
- If we both send and receive, we can use checker circuit as generator.
  - Set the P input as 0. $x \oplus y \oplus z \oplus 0 = x \oplus y \oplus z$




(a) 3-bit even-parity generator




(b) 4-bit even-parity checker